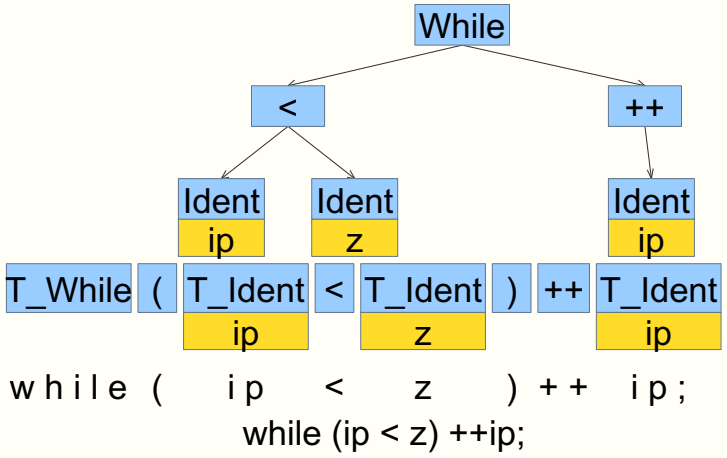
《编译原理》

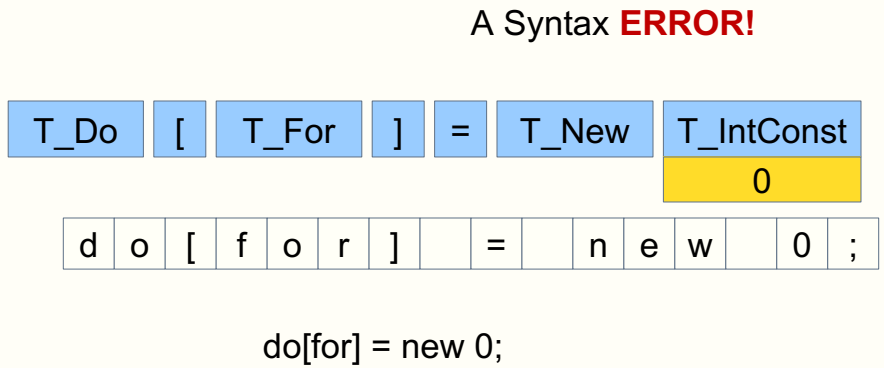
第12讲：语法分析

何冬杰
重庆大学

What is Syntax Analysis?

- After lexical analysis (scanning), we have a series of tokens
- In syntax analysis (or parsing), we interpret what those tokens mean
- Goal: recover the ***structure*** described by that series of tokens
  - E.g., Parse Tree/AST
  - Report ***errors*** if those tokens do not properly encode a structure.





Review: What Have We Learned?

- CFG: a set of productions of the form

  symbol → symbol ... symbol

- The toy language **LANG** using CFG grammar

```
program     ::= decl* EOF
decl        ::= class_decl | func_decl | global_decl
class_decl  ::= 'class' ID ('extends' ID)?
                '{' (field_decl | method_decl)* '}'
field_decl  ::= type ID ';'
method_decl ::= type ID '(' param_list? ')' '{' stmt* '}'
func_decl   ::= method_decl  ……
```

```
stmt ::= ';'
   | '{' stmt* '}'
   | local_decl
   | 'if' '(' exp ')' stmt ('else' stmt)?
   | 'while' '(' exp ')' stmt
   | 'return' exp? ';'
   ……
```

- The AST structure of LANG
- Use ***Menhir*** to generate a **parser**
- Following lectures: learn how to **write a parser** by hand

Derivations and Parse Tree

- To show that a sentence is in the language of a grammar, we can perform a **derivation**
  - Start with the start symbol; repeatedly replace a non-terminal with its RHS (Right-Hand Side)
  - **Leftmost derivation**: in each step, expands the leftmost nonterminal.
  - **Rightmost derivation**: expands the rightmost one

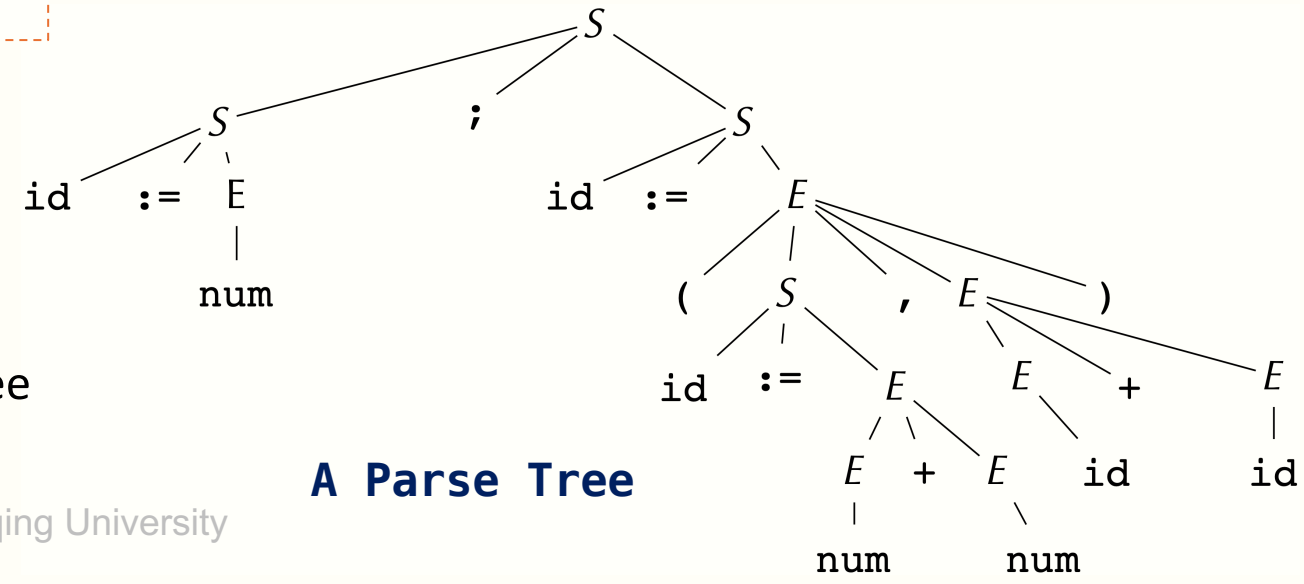
$S \rightarrow S\ \texttt{;}\ S$
$S \rightarrow \texttt{id}\ \texttt{:=}\ E$
$S \rightarrow \texttt{print}\ \texttt{(}\ L\ \texttt{)}$
$E \rightarrow \texttt{id}$
$E \rightarrow \texttt{num}$
$E \rightarrow E + E$
$E \rightarrow \texttt{(}S\texttt{,}\ E\texttt{)}$
$L \rightarrow E$
$L \rightarrow L\texttt{,}\ E$

Grammar

A Derivation

$$
\begin{aligned}
S &\rightarrow S; S \\
&\rightarrow id := E; S \\
&\rightarrow id := num;\ S \\
&\rightarrow id := num; id := E \\
&\rightarrow \ \ldots \\
&\rightarrow \text{id} := \text{num};\ \text{id} := (\text{id} := \text{num} + \text{num}, \text{id} + \text{id})
\end{aligned}
$$

- A derivation constructs a **parse tree**
  - Also called **a syntax tree**
  - Different derivations may have the same tree

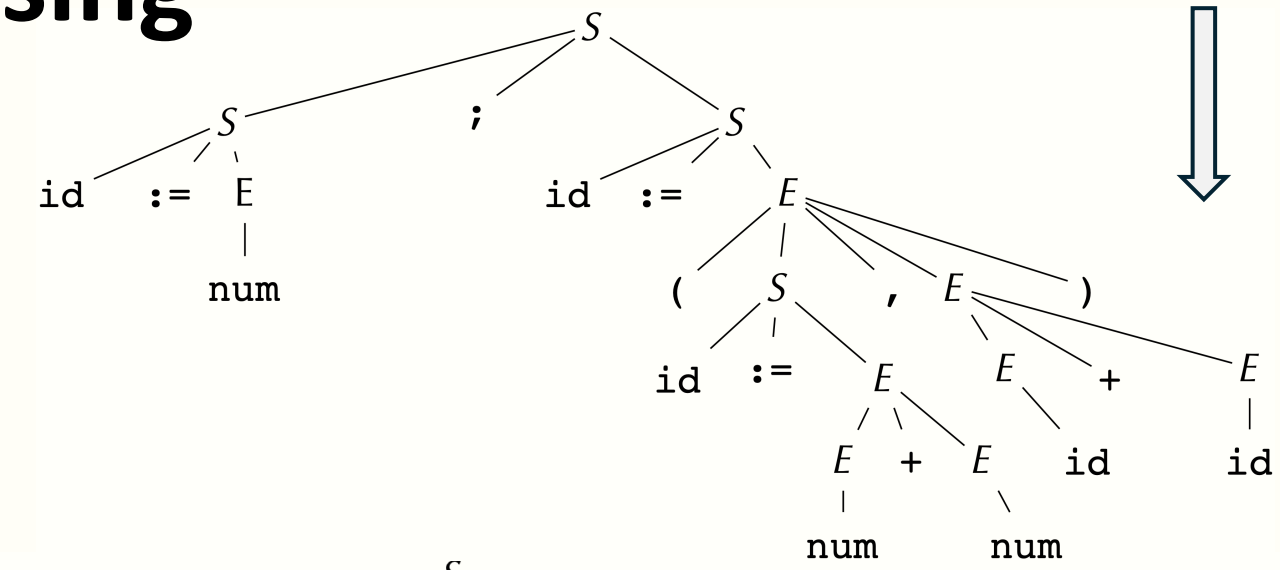


A Parse Tree

Different Types of Parsing

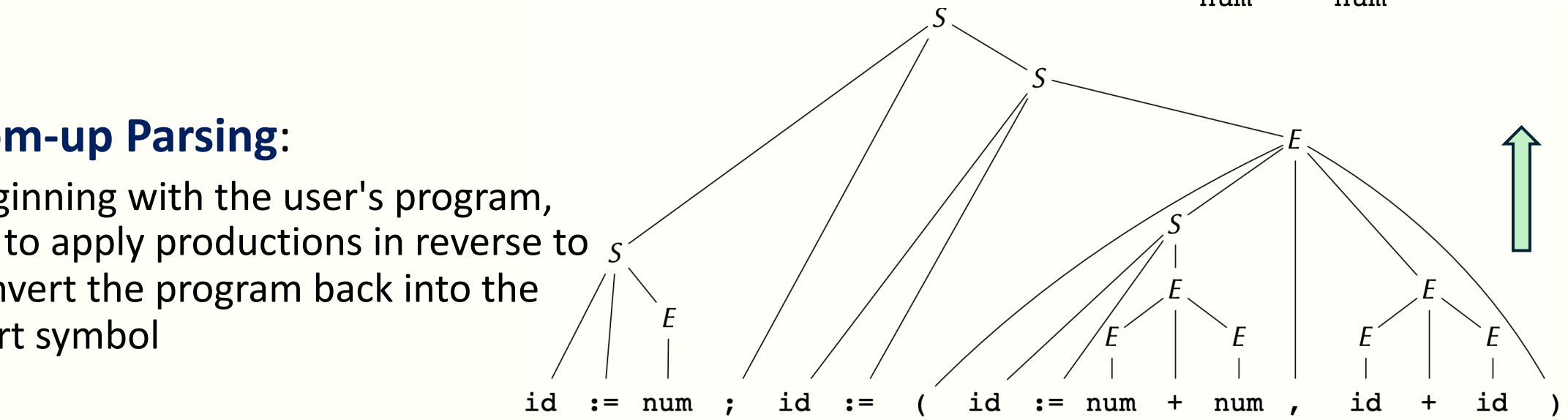
❑Top-down Parsing:

- Beginning with the start symbol, try to guess the productions to apply to end up at the user's program.



❑Bottom-up Parsing:

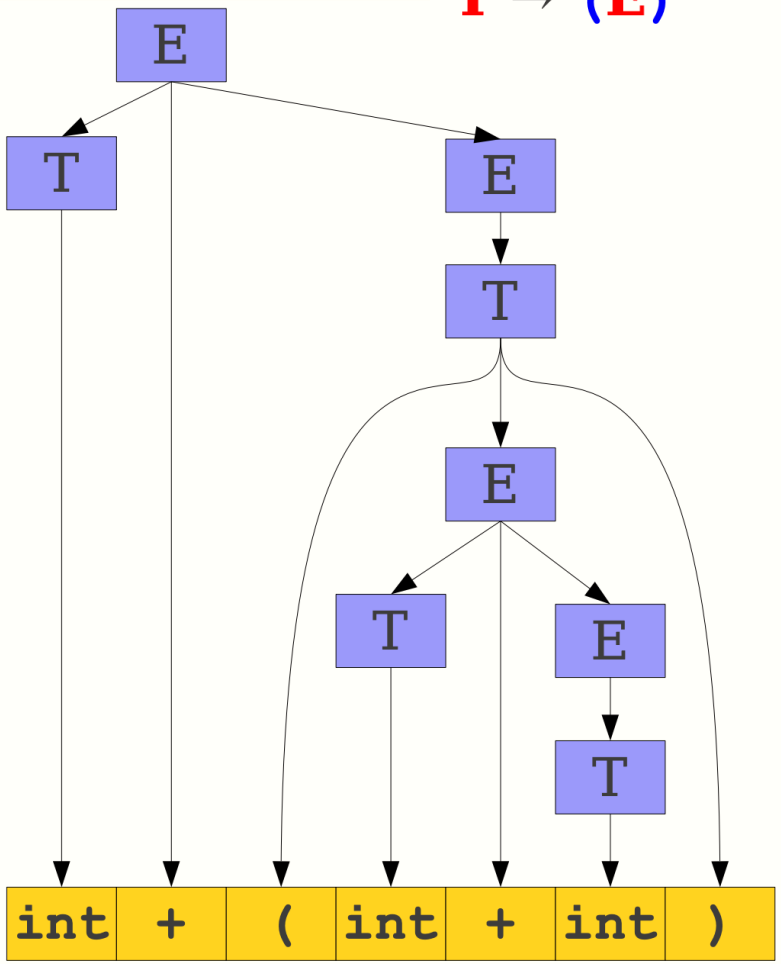
- Beginning with the user's program, try to apply productions in reverse to convert the program back into the start symbol



Challenges in Top-Down Parsing

$$E \rightarrow T$$
$$E \rightarrow T + E$$
$$T \rightarrow \texttt{int}$$
$$T \rightarrow (E)$$

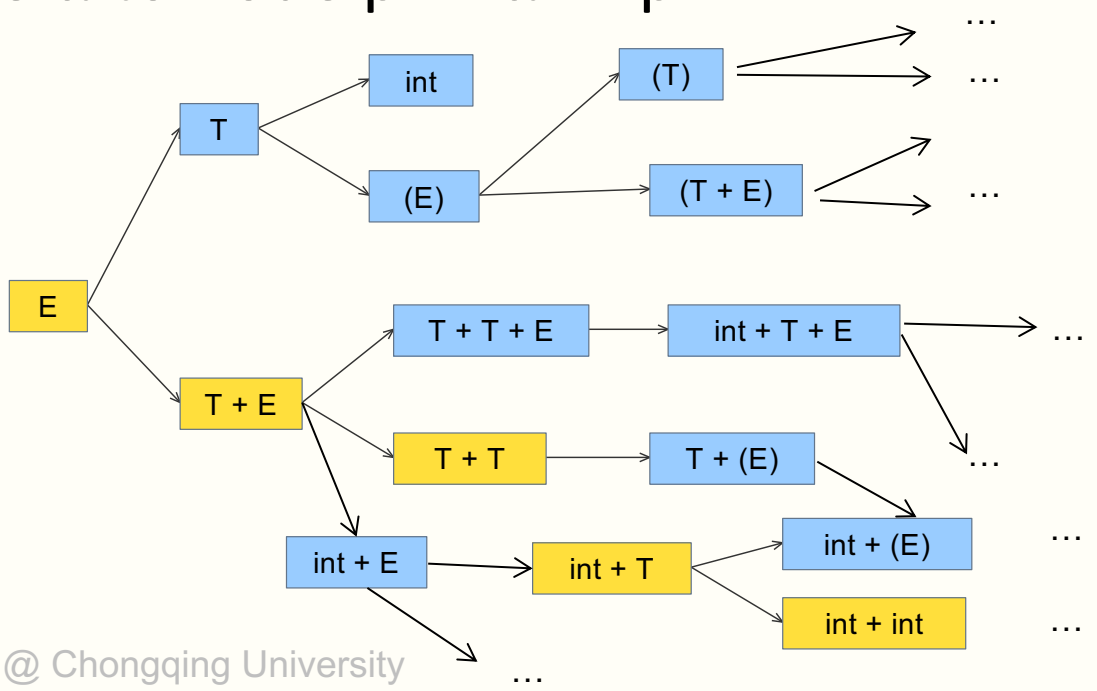
- Top-down parsing begins with virtually no information.
  - Begins with just the start symbol, which matches every program.
- How can we know which productions to apply?
- In general, we can't.
  - There are some grammars for which the best we can do is guess and backtrack if we're wrong.
  - If we have to guess, how do we do it?



Parsing as a Search

- An idea: treat parsing as a graph search: **BFS, DFS**
- Each node is a **sentential form**
  - a string of terminals and nonterminals derivable from the start symbol
- There is an edge from node α to node β iff α ⇒ β .

$E \rightarrow T$
$E \rightarrow T + E$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$



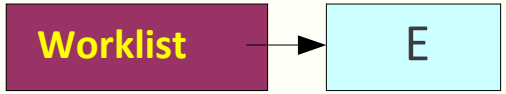
Problems of blind searching

- Worst-case runtime is **exponential**
- Worst-case memory usage is **exponential** (for BFS)

- **Can we do better?**

- In top-down parsing,
  - We have the token streams as the input
  - Pruning: we can perform the **leftmost derivations** and check if the **prefix** of derivations matches the input
  - Left BFS and Left DFS

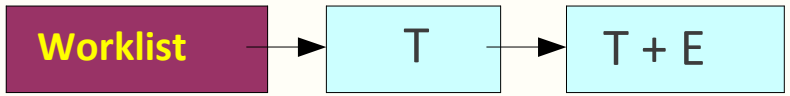
Leftmost BFS

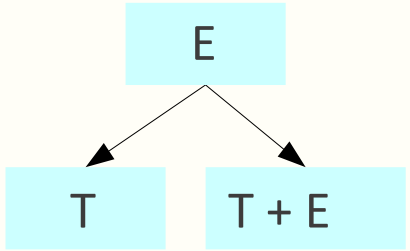


E → T
E → T + E
T → int
T → (E)

int + int

Leftmost BFS





E → T

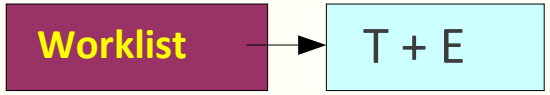
E → T + E

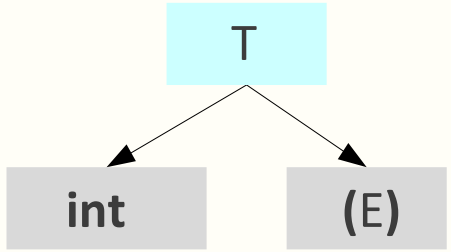
T → int

T → (E)

int + int

Leftmost BFS





E → T

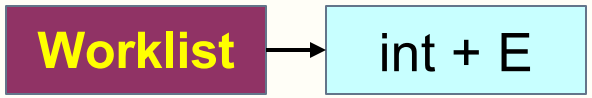
E → T + E

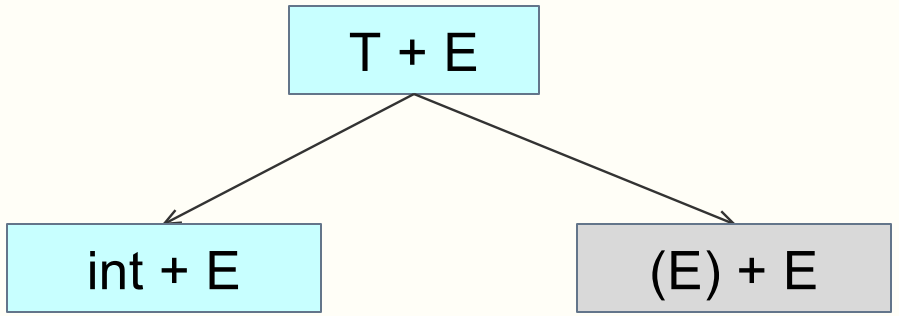
T → int

T → (E)

int + int

Leftmost BFS





E → T

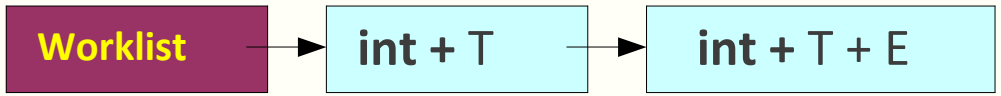
E → T + E

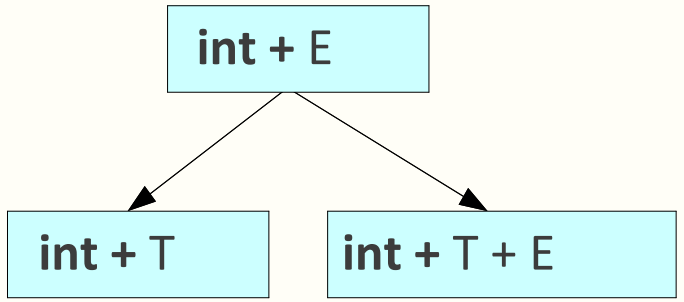
T → int

T → (E)

int + int

Leftmost BFS





E → T

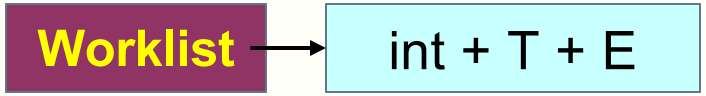
E → T + E

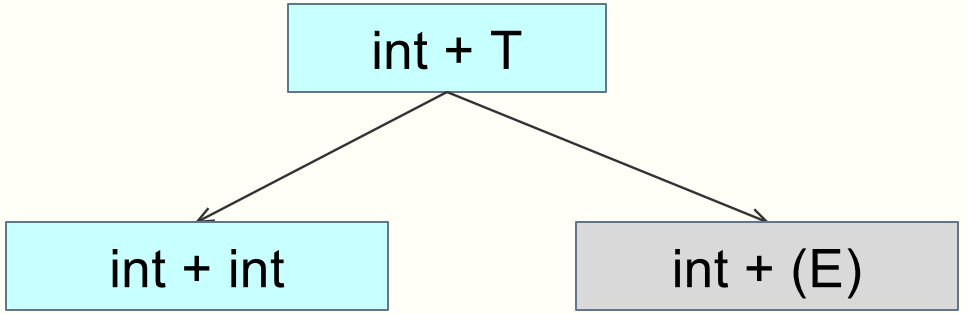
T → int

T → (E)

int + int

Leftmost BFS





E → T

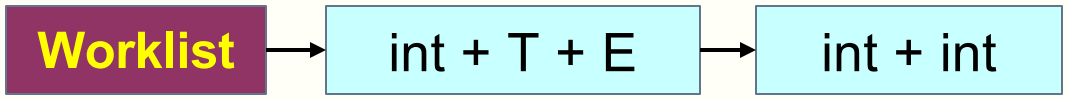
E → T + E

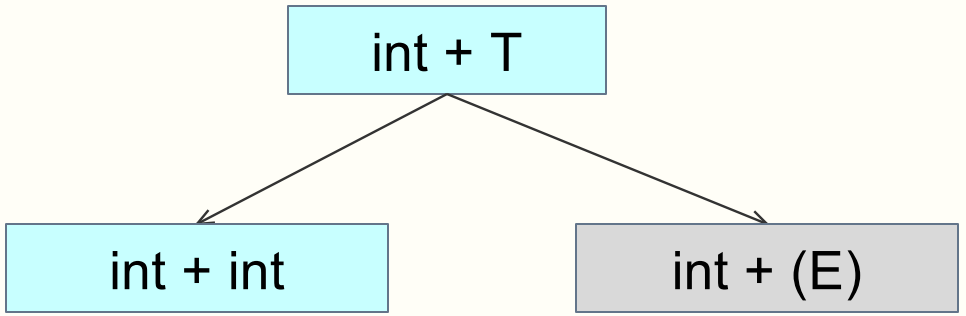
T → int

T → (E)

int + int

Leftmost BFS





E → T

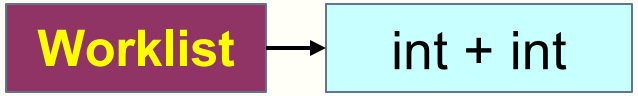
E → T + E

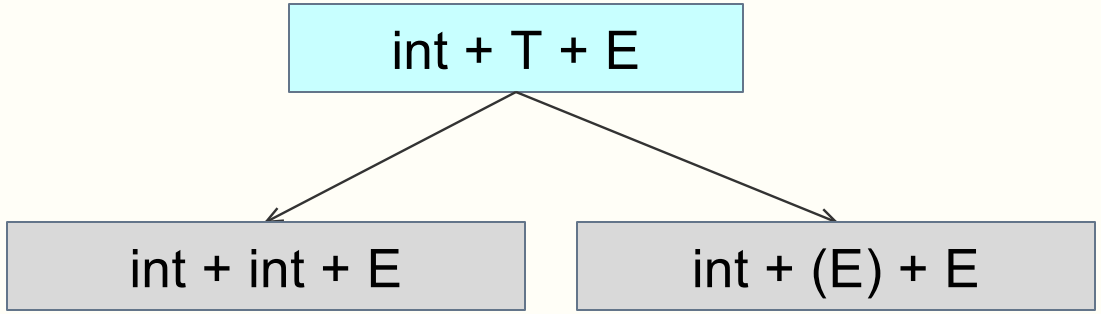
T → int

T → (E)

int + int

Leftmost BFS





E → T

E → T + E

T → int

T → (E)

int + int

Leftmost BFS

Worklist



int + int

E → T

E → T + E

T → int

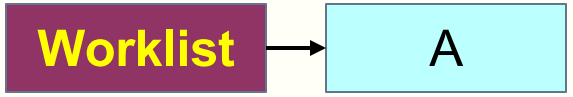
T → (E)

int + int

Leftmost BFS

- Substantial improvement over naïve algorithm (i.e., BFS).
- Will always find a valid parse of a program if one exists.
- Can easily be modified to find if a program can't be parsed.
- But there are still problems.

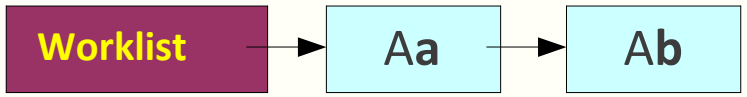
Leftmost BFS Has Problems

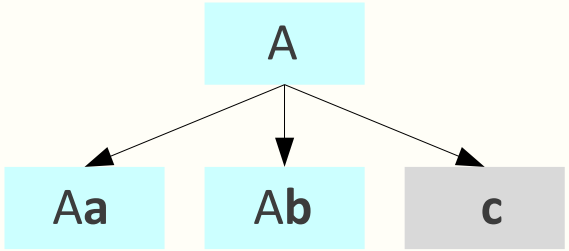


$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

Leftmost BFS Has Problems





A → Aa | Ab | c

caaaaaaaaaa

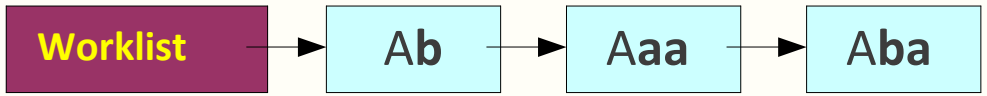
Leftmost BFS Has Problems

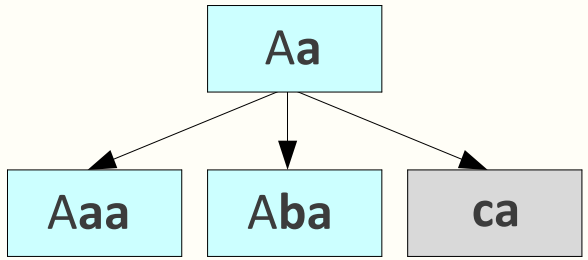


$A \rightarrow Aa \mid Ab \mid c$

caaaaaaaaaa

Leftmost BFS Has Problems

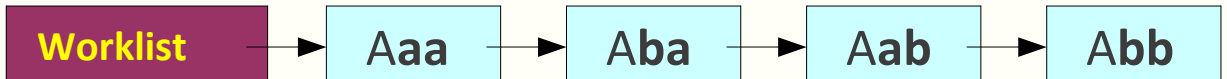


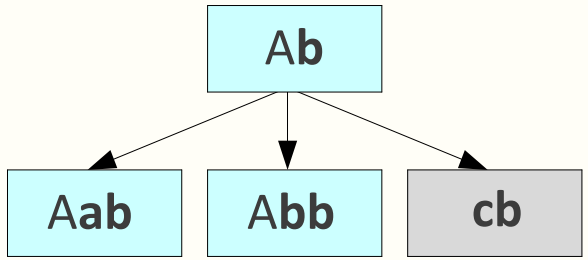


A → Aa | Ab | c

caaaaaaaaaa

Leftmost BFS Has Problems





A → Aa | Ab | c

caaaaaaaaaa

Leftmost BFS Has Problems



A → **Aa** | **Ab** | **c**

caaaaaaaaaa

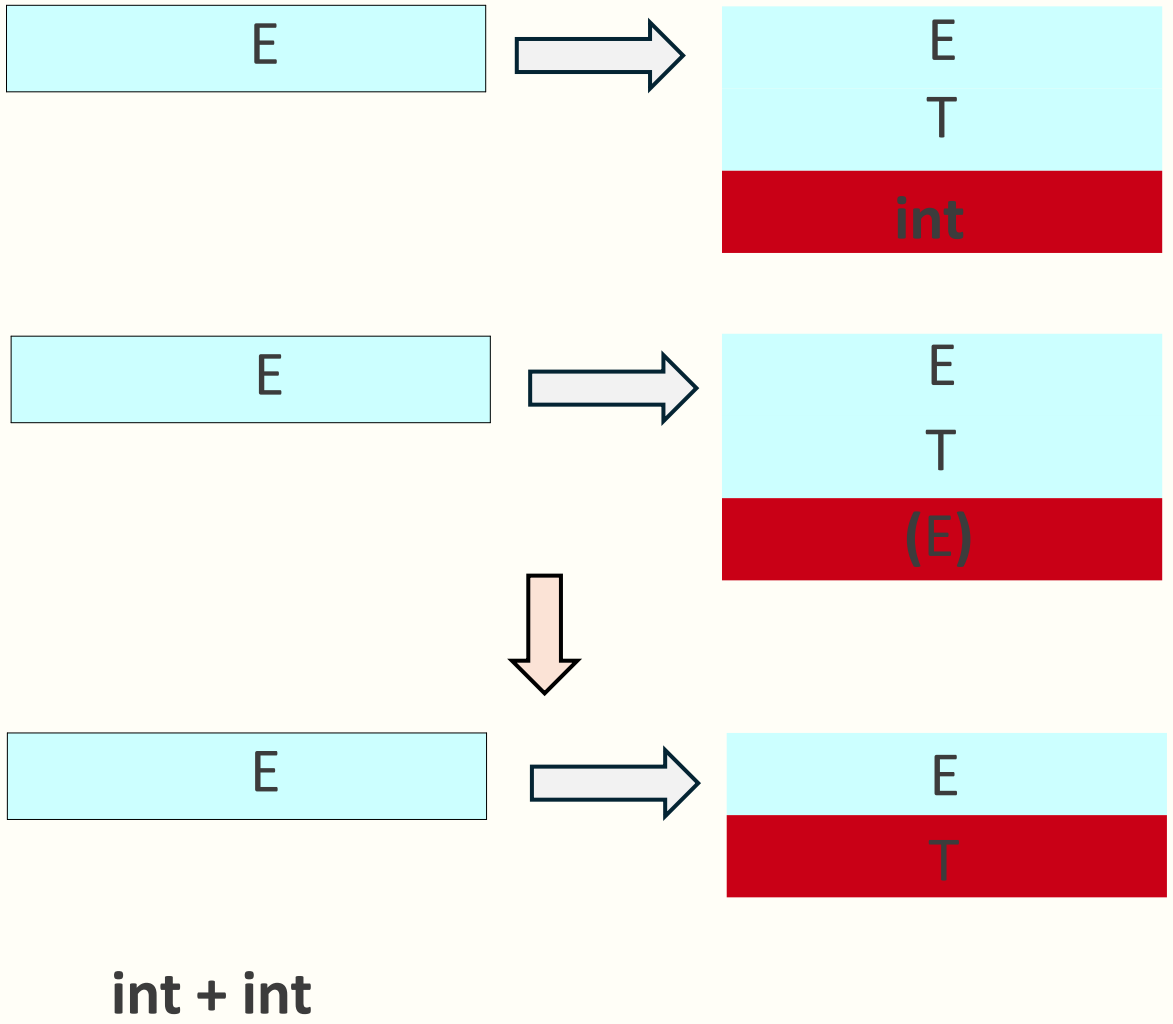
Problems with Leftmost BFS

- Grammars like this can make parsing take **exponential** time.
- Also uses **exponential** memory.
- What if we search the graph with a different algorithm?

- **Leftmost DFS:**
  - **Idea**: Use depth-first search and leftmost derivation
  - Advantages:
    - **Lower** memory usage: Only considers one branch at a time.
    - **High** performance: On many grammars, runs very quickly.
    - **Easy** to implement: Can be written as a set of mutually recursive functions.

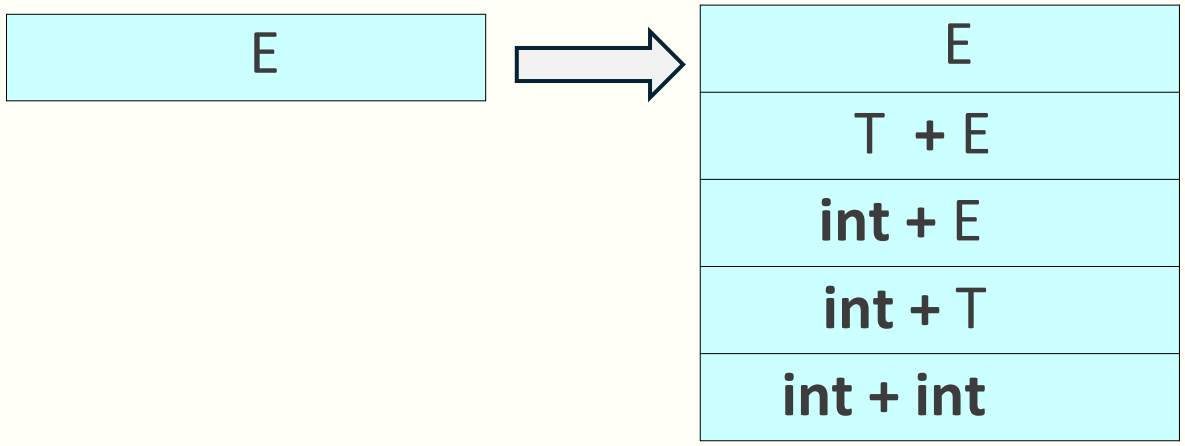
Leftmost DFS

E → T
E → T + E
T → int
T → (E)



int + int

Leftmost DFS



E → T
E → T + E
T → int
T → (E)

int + int



Problems with Leftmost DFS

$A \rightarrow Aa \mid c$

A
Aa
Aaa
Aaaa
Aaaaa



- Left Recursion
  - A nonterminal **A** is said to be **left-recursive** iff $A \Rightarrow^* A\omega$ for some string ω.
  - Leftmost DFS may fail on left-recursive grammars.
  - Fortunately, in many cases it is possible to eliminate left recursion.
    - Example: rewrite the grammar to eliminate left recursion.

$E \rightarrow E + T$	$T \rightarrow T * F$	$F \rightarrow id$
$E \rightarrow E - T$	$T \rightarrow T / F$	$F \rightarrow num$
$E \rightarrow T$	$T \rightarrow F$	$F \rightarrow (E)$

⇒

$E \rightarrow T\ E'$	$T \rightarrow F\ T'$	$F \rightarrow id$
$E' \rightarrow +\ T\ E'$	$T' \rightarrow *\ F\ T'$	$F \rightarrow num$
$E' \rightarrow -\ T\ E'$	$T' \rightarrow /\ F\ T'$	$F \rightarrow (E)$
$E' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$	

Summary of Leftmost BFS/DFS

- Leftmost BFS works on all grammars.
  - Worst-case runtime is exponential.
  - Worst-case memory usage is exponential.
  - Rarely used in practice.
- Leftmost DFS works on grammars without left recursion.
  - Worst-case runtime is exponential.
  - Worst-case memory usage is linear.
  - Often used in a limited form as **recursive descent**.

Recursive Descent Parsing

- More direct implementation of grammar
  - Each non-terminal is a function
  - Easy to write and diagnose, but hard to maintain.
- Example: recursive descent parsing with **backtracking**
  - Essentially an implementation of the leftmost DFS

S → aA | aB
A → b
B → c

```
func parseA():
    return match('b')

func parseB():
    return match('c')
```

```
func parseS():
    start = pos
    // Try S → aA
    if match('a') and parseA():
        return SUCCESS
    pos = start     // Backtrack!
    // Try S → aB
    if match('a') and parseB():
        return SUCCESS
    pos = start
    return FAILURE
```

Recursive Descent Parsing without Backtracking

- Example: a **recursive descent** parser for

Expr := INT | ID | LPAREN Expr RPAREN | LET ID EQ Expr IN Expr

```
let rec parse_expr () =
  match !lookahead with
  | INT -> match_token INT
  | ID -> match_token ID
  | LPAREN -> match_token LPAREN;
      parse_expr (); match_token RPAREN
  | LET -> match_token LET; match_token ID; match_token EQ;
      parse_expr (); match_token IN; parse_expr ()
  | _ -> raise ParseError
```

```
let match_token t =
  if !lookahead = t then
    lookahead := next_token ()
  else raise ParseError
```

- Exploiting **Lookahead** to choose productions
  - Essentially an implementation of LL(k), a kind of **predictive parsing**

Summary

- Top-down Parsing
  - Leftmost search explores derivations with BFS or DFS
  - Recursive descent with backtracking follows leftmost DFS
  - LL(k) can be implemented as recursive descent without backtracking
  - Lookahead determines which production to use (next lecture)
- Bottom-up Parsing
  - LR(k) builds the parse through shift and reduce actions (the lecture after next)